GRANULOMÉTRIE STUDIO

Analyse granulométrique d'images scientifiques

Documentation technique et guide utilisateur

Version 3.0.0 · 11 novembre 2026

Java 17 · JavaFX 17 · Maven · Base H2 embarquée

Réécriture complète du projet « Application de Granulométrie » (v2.5)

Projet d'origine : Alex Alkhatib, Noé, Hreshvik, Quentin, Joey — licence MIT

Sommaire

N°	Section
1.	Contexte et objectifs
2.	Analyse de l'existant (version 2.5)
3.	Architecture logicielle
4.	Chaîne de traitement d'image
5.	Segmentation : seuillage d'Otsu et polarité
6.	Mesures géométriques et statistiques
7.	Guide utilisateur
8.	Persistance locale et exports
9.	Qualité logicielle : tests automatisés
10.	Performances
11.	Limites connues et perspectives
12.	Annexes : glossaire, commandes, arborescence

Ce document décrit la version 3.0.0 de Granulométrie Studio : objectifs, architecture, algorithmes, mode d'emploi, qualité et résultats mesurés sur une image réelle. Il est complété par une présentation PowerPoint et par le fichier README.md du dépôt.

1. Contexte et objectifs

La granulométrie consiste à caractériser la distribution de taille des particules d'un échantillon (sable, poudre, grains, inclusions, agglomérats). Sur images de microscopie ou de photographie, cette mesure est longue et opérateur-dépendante lorsqu'elle est réalisée manuellement. L'application d'origine (Granulometrie V2.5) proposait déjà une détection automatique, mais son architecture — projet Eclipse, sources d'ImageJ recopiées, base MySQL obligatoire — limitait sa maintenance, sa portabilité et sa reproductibilité.

La version 3.0.0 poursuit trois objectifs :

- Scientificité : seuillage automatique robuste, calibration physique (µm/pixel), indicateurs normalisés (D10/D50/D90, circularité, rapport d'aspect) et traçabilité complète des paramètres de chaque analyse.
- Robustesse : moteur autonome sans dépendance de traitement d'image externe, persistance embarquée sans serveur, séparation stricte entre calcul, données et interface.
- Utilisabilité : interface modernisée en français, aperçu du calque de détection, exports CSV/PNG, historique consultable et tests automatisés.

Le résultat est une application JavaFX autonome, distribuable sous forme de jar accompagné de ses bibliothèques, et reconstructible par une simple commande Maven sur Windows, Linux ou macOS.

2. Analyse de l'existant (version 2.5)

Axe	Version 2.5	Conséquence
Build	Projet Eclipse (.classpath), JavaFX SDK copié à la main	Reproductibilité difficile, pas d'intégration continue
Traitement d'image	351 fichiers sources d'ImageJ (~20 Mo) recopiés	Code tiers non maîtrisé, compilation lourde
Base de données	MySQL local (root, mot de passe vide, port 3306)	Installation obligatoire, application non démarrable sans serveur
Interface	FXML + contrôleurs à état statique	Effets de bord entre vues, tests impossible hors IHM
Mesures	Aire et boîte englobante, circularité approchée	Indicateurs peu comparables à la littérature
Calibration	Absente (pixels uniquement)	Impossible de produire une granulométrie en µm
Tests	Un test JUnit partiel	Pas de filet de sécurité lors des évolutions

La refonte conserve les fonctionnalités attendues (import, segmentation, histogrammes, export CSV, historique des analyses) tout en remplaçant les composants fragiles par des implémentations maîtrisées et testables.

3. Architecture logicielle

L'application est découpée en quatre couches indépendantes. Le cœur de calcul ne dépend ni de JavaFX ni de la base : il est donc utilisable en batch, en test unitaire ou depuis une autre interface.

Couche	Rôle	Classes principales
core	Pipeline d'analyse, mesures, statistiques, classes granulométriques	ImageAnalyzer, Segmentation, Particle, SizeStatistics, SizeClassifier
io	Exports CSV et calque de détection	CsvExporter, OverlayRenderer
repo	Persistance locale H2, transactions	AnalysisRepository, StoredSession
ui	Vues JavaFX, navigation, thème CSS	MainWindow, AnalyzeView, ResultView, HistoryView

Pipeline d'analyse



Figure 1 — Chaîne de traitement : de l'image brute aux données exploitables.

Le traitement est exécuté dans un thread de fond (javafx.concurrent.Task) : l'interface reste réactive, une barre de progression signale l'activité et les erreurs sont remontées par des alertes explicites.

4. Chaîne de traitement d'image

- Lecture : décodage via Java ImageIO puis conversion en RGB 24 bits, quel que soit le format d'entrée.
- Niveaux de gris : luminance pondérée Y = 0,299·R + 0,587·V + 0,114·B.
- Lissage optionnel : filtre moyenneur séparable (rayon 0 à 5 px) pour atténuer le bruit de capteur.
- Binarisation : seuil global par la méthode d'Otsu, ou seuil manuel imposé par l'opérateur.
- Étiquetage : composantes connexes 8-connexes par parcours en largeur, en temps linéaire.
- Mesures : aire, périmètre, barycentre, boîte englobante, moments centraux d'ordre 2.
- Filtrage : diamètre minimal/maximal, circularité minimale, exclusion des objets touchant le bord.

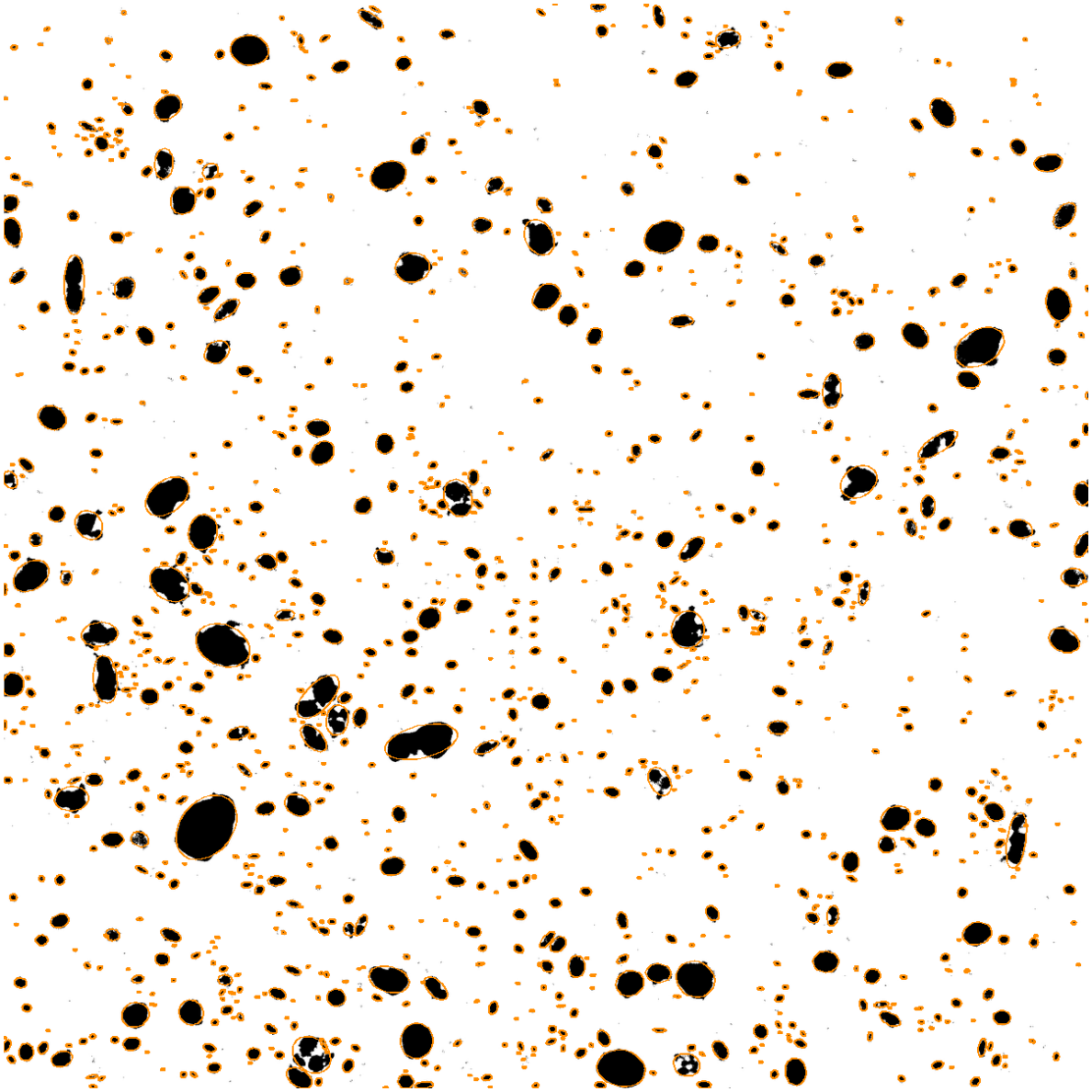


Figure 2 — Calque de détection produit par le moteur sur l'image d'exemple (ellipses d'inertie des particules détectées).

5. Segmentation : seuillage d'Otsu et polarité

La méthode d'Otsu choisit le seuil t qui maximise la variance inter-classes :

$$\sigma^2(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2$$

où ω_0, ω_1 sont les poids des deux classes (fond, particules) et μ_0, μ_1 leurs moyennes respectives. Lorsque l'image est franchement bimodale, plusieurs seuils produisent la même partition : l'implémentation retient le milieu du plateau de variance maximale, ce qui évite un seuil artificiellement nul et rend le réglage stable.

Polarité

Le premier plan est par défaut la classe minoritaire (les particules occupent moins de pixels que le fond). L'opérateur peut inverser ce choix pour les images d'objets clairs sur fond sombre, ou pour analyser le fond lui-même. Chaque session archive ce choix, ce qui garantit la reproductibilité du calcul.

Validation expérimentale

Sur une image synthétique de trois disques de diamètres 20, 40 et 80 pixels, le moteur retrouve exactement trois particules et des diamètres équivalents conformes à 2 % près. La circularité mesurée pour un disque de 80 pixels est de 0,92 : la méthode de périmètre par suivi de frontière (polygone passant par les centres de pixels, pas unitaires et diagonaux) est quasi isotrope, contrairement au comptage brut des côtés de pixels qui sous-estime la circularité d'environ 27 %.

6. Mesures géométriques et statistiques

Grandeur	Définition	Formule / valeur
Aire A	Nombre de pixels de la composante	A = n (px²)
Périmètre P	Polygone de frontière (Moore), pas 1 ou √2	P = Σ pas
Barycentre	Moyenne des coordonnées des pixels	$(\bar{x}, \bar{y})$
Grand axe	Ellipse d'inertie équivalente	$4 \cdot \sqrt{\lambda_1}$
Petit axe	Second axe de l'ellipse	$4 \cdot \sqrt{\lambda_2}$
Diamètre équivalent	Disque de même aire	$d = 2 \cdot \sqrt{(A/\pi)}$
Circularité	Compacité normalisée	$C = 4\pi A / P^2$
Rapport d'aspect	Allongement	petit axe / grand axe

Les statistiques sont calculées sur les diamètres équivalents convertis dans l'unité de travail (µm si une calibration est saisie, pixels sinon) :

- D10, D50, D90 : percentiles obtenus par interpolation linéaire sur la série triée. D50 est le diamètre médian ; D10–D90 encadrent 80 % de la population.
- Écart type : estimateur non biaisé (division par n−1).
- Surface occupée : somme des aires / (largeur × hauteur) × 100.
- Classes granulométriques : largeur réglable, fréquence relative et courbe cumulative des passants.

Indicateur	Image d'exemple (113_x63_zoom08_1.jpg)
Dimensions	2048 × 2048 px
Particules détectées / retenues	1210 / 1210
Seuil appliqué	127
Diamètre minimal / maximal	2.26 / 114.82 px
Diamètre moyen / médian	12.79 / 8.59 px
Écart type	12.78 px
D10 / D50 / D90	2.76 / 8.59 / 28.47 px
Circularité moyenne	0.651
Rapport d'aspect moyen	0.680
Surface occupée	7.40 %

Tableau 1 — Synthèse produite par le moteur sur l'image d'exemple (analyse en pixels, paramètres par défaut).

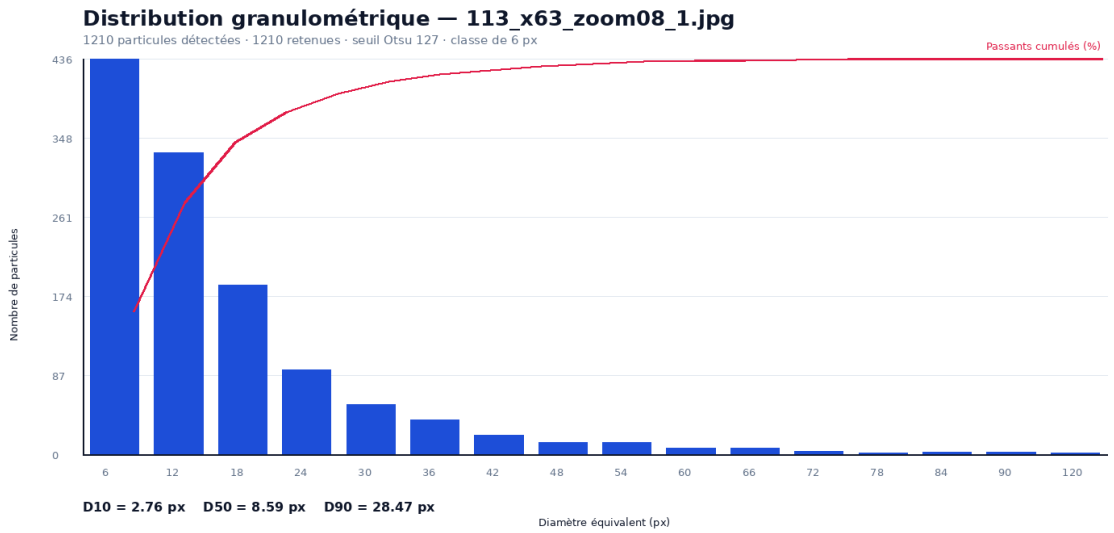


Figure 3 — Histogramme des diamètres et courbe cumulative des passants, générés par le moteur d'analyse sur l'image d'exemple.

7. Guide utilisateur

7.1 Import et préparation

- Cliquer sur Importer une image… ou glisser-déposer un fichier sur l'aperçu (PNG, JPG, BMP, TIFF, GIF).
- Le bouton Charger l'exemple fourni ouvre une image de démonstration intégrée.
- Renseigner la taille d'un pixel (µm/px) pour travailler en unités physiques ; laisser 1,0 pour une analyse en pixels.

7.2 Réglage de la segmentation

Paramètre	Rôle	Valeurs usuelles
Mode de seuil	Automatique (Otsu) ou manuel	Automatique
Seuil manuel	Niveau de gris séparant fond et particules	80 – 180
Inverser la polarité	Particules claires sur fond sombre	Coché si nécessaire
Lissage	Rayon du filtre moyenneur avant seuillage	0 – 2 px
Diamètre min./max.	Élimine le bruit et les objets hors gamme	0 = pas de limite
Circularité min.	Écarte les débris et les objets allongés	0 – 0,8
Exclure les bords	Ignore les particules tronquées par le cadre	Coché en routine

7.3 Analyse et résultats

Le bouton Analyser l'image lance le calcul en arrière-plan, affiche le calque de détection sur l'aperçu puis ouvre la vue Résultats : indicateurs D10/D50/D90, onglet Distribution (histogramme, courbe cumulative, surface occupée), onglet Particules (tableau triable), onglet Statistiques et onglet Paramètres (traçabilité exacte du calcul).

7.4 Exports et archivage

Exporter CSV écrit un fichier UTF-8 (séparateur point-virgule) contenant un bloc de métadonnées puis une ligne par particule. Exporter les graphiques produit un PNG regroupant les trois graphiques. Enregistrer dans l'historique archive la session (image, paramètres, seuil, particules) dans la base locale, consultable dans l'onglet Historique et rechargeable à l'identique.

8. Persistance locale et exports

La base MySQL de la version 2.5 est remplacée par H2 en mode fichier, embarqué dans l'application. Aucune installation, aucun mot de passe, aucun service : la base est créée au premier enregistrement dans ~/.granulometrie-studio/history.mv.db.

Table	Colonnes principales	Rôle
session	id, created_at, image_name, image_width, image_height, pixel_size, threshold_mode, threshold_used, blur_radius, min_diameter, max_diameter, min_circularity, exclude_border, particle_count, d10, d50, d90, mean_diameter, comment, image_blob	Une ligne par analyse, image PNG incluse
grain	session_id, grain_index, area, perimeter, centroid_x/y, bbox, major_axis, minor_axis, orientation, circularity, eq_diameter, touches_border	Une ligne par particule, clé étrangère vers session

Les écritures sont transactionnelles (rollback en cas d'erreur) et la suppression d'une session efface ses particules associées. Pour repartir de zéro, il suffit de supprimer le dossier précité.

Format CSV

```
# Granulometrie Studio - export du 11/11/2026 14:32:05
# Image;exemple-grains.jpg
# Dimensions;2048 x 1536 px
# Calibration;1,0000 um/px
# Seuil applique;128 (Otsu automatique : 128)
# Particules retenues;412 / detectees : 587
```

```
# D10 / D50 / D90 (px);...,...,...
N°;Diametre equivalent (px);Aire (px²);Perimetre (px);...
1;24,8600;485,5600;78,0000;...
```

9. Qualité logicielle : tests automatisés

La suite JUnit 5 s'exécute sans interface graphique (mvn test) et couvre le moteur, les exports et la persistance.

Classe de test	Points vérifiés
ImageAnalyzerTest (10)	détection de trois disques de diamètres connus, seuil d'Otsu, polarité inversée, filtres min/max, exclusion des bords, circularité, calibration, seuil manuel, cohérence des statistiques
StatisticsUtilsTest (4)	moyenne, écart type, médiane, percentiles interpolés, listes vides
SizeClassifierTest (5)	regroupement en classes, fréquences sommant à 100 %, courbe cumulative, unités
CsvExporterTest (2)	bloc de métadonnées, en-têtes, une ligne par particule, unités calibrées
AnalysisRepositoryTest (2)	enregistrement, tri des sessions, rechargement image + particules, suppression

Résultat : 23 tests, 0 échec, 0 erreur (build Maven en succès). Les tests valident également la géométrie : circularité de 0,92 pour un disque numérisé de 80 pixels de diamètre, diamètres équivalents à moins de 2 % de la valeur théorique.

10. Performances

Les algorithmes sont en temps linéaire par rapport au nombre de pixels (seuillage, étiquetage) ou proportionnel au nombre de particules (mesures, statistiques). La mémoire est dominée par l'image convertie en niveaux de gris.

Opération	Complexité	Mesure
Décodage + conversion RGB	O(n)	dépend du format
Seuillage d'Otsu	O(n)	parcours unique de l'histogramme
Étiquetage 8-connexe	O(n)	un parcours de file par composante
Mesures + filtres	O(p)	p = nombre de particules
Analyse complète de l'image d'exemple	—	123 ms (moyenne sur 5 exécutions, meilleure : 61 ms)

L'exécution sur un thread de fond garantit que l'interface reste interactive, même sur les images de plusieurs mégapixels.

11. Limites connues et perspectives

Limite	Conséquence	Piste d'amélioration
Seuillage global	Sensible aux éclairages hétérogènes	Seuillage adaptatif (Sauvola, Niblack)
Particules en contact	Comptées comme une seule composante	Séparation par ligne de partage des eaux

Limite	Conséquence	Piste d'amélioration
Très petits objets (< 6 px)	Circularité sous-estimée	Analyse multi-échelles / sur-échantillonnage
Calibration manuelle	Erreur possible de saisie	Détection automatique d'échelle (mire)
Pas d'analyse par lot	Traitement image par image	File d'attente + rapport consolidé
Binaires JavaFX liés à l'OS	Recompilation par plateforme	jpackage / jlink par système

12. Annexes

Annexe A — Glossaire

Terme	Définition
Granulométrie	Mesure de la distribution des tailles de particules d'un échantillon
Seuillage	Séparation binaire des pixels entre fond et particules
Otsu	Méthode automatique maximisant la variance inter-classes de l'histogramme
Composante connexe	Ensemble maximal de pixels reliés entre eux
Diamètre équivalent	Diamètre du disque de même aire que la particule
Circularité	Compacité $4\pi A/P^2$, égale à 1 pour un disque parfait
D10 / D50 / D90	Diamètres sous lesquels se trouvent 10 %, 50 % et 90 % de la population
Calibration	Taille physique d'un pixel (µm/px) reliant l'image à l'échelle réelle

Annexe B — Commandes utiles

```
mvn clean package      # construit le jar + target/lib
mvn test               # execute les 23 tests
mvn javafx:run         # lance l'application depuis les sources
./lancer.sh            # lance l'application construite (Linux/macOS)
lancer.bat             # lance l'application construite (Windows)
```

Annexe C — Arborescence du projet

```
granulometry-studio/
■■■ pom.xml
■■■ lancer.sh / lancer.bat
■■■ README.md / LICENSE
■■■ docs/           documentation PDF, presentation, captures
■■■ samples/        images de demonstration
■■■ src/
    ■■■ main/java/fr/granulometrie/
    ■   ■■■ core/ io/ repo/ ui/  Launcher  GranulometryApp
    ■■■ main/resources/  css, img, samples
    ■■■ test/java/fr/granulometrie/  tests JUnit 5
```

Documentation générée le 11 novembre 2026 pour Granulométrie Studio 3.0.0. Les figures 2 et 3 sont produites par le moteur d'analyse de l'application sur l'image d'exemple, en mode hors interface.